LangChain Documentation: Retrieval & RAG

Source: https://docs.langchain.com/oss/python/langchain/retrieval

Large Language Models (LLMs) are powerful, but they have two key limitations:

- **Finite context** — they can't ingest entire corpora at once.
- **Static knowledge** — their training data is frozen at a point in time.

Retrieval addresses these problems by fetching relevant external knowledge at query time. This is the foundation of **Retrieval-Augmented Generation (RAG)**: enhancing an LLM's answers with context-specific information.

Building a Knowledge Base

A knowledge base is a repository of documents or structured data used during retrieval. If you need a custom knowledge base, you can use LangChain's document loaders and vector stores to build one from your own data.

You can build a searchable knowledge base from your own data using LangChain's document loaders, embeddings, and vector stores — for example, building a search engine over a PDF that retrieves passages relevant to a query, and implementing a minimal RAG workflow on top of this engine.

From Retrieval to RAG

Retrieval allows LLMs to access relevant context at runtime. Most real-world applications go one step further: they integrate retrieval with generation to produce grounded, context-aware answers. This is the core idea behind Retrieval-Augmented Generation (RAG). The retrieval pipeline becomes a foundation for a broader system that combines search with generation.

Retrieval Pipeline

A typical retrieval workflow looks like this:

```
Sources (Google Drive, Slack, Notion, etc.) --> Document Loaders --> Documents
Documents --> Split into chunks --> Turn into embeddings --> Vector Store
User Query --> Query embedding --> Vector Store --> Retriever --> LLM uses retrieved info --> Answer
```

Each component is modular: you can swap loaders, splitters, embeddings, or vector stores without rewriting the app's logic.

Building Blocks

- **Document loaders**: Ingest data from external sources (Google Drive, Slack, Notion, PDFs, etc.), returning standardized `Document` objects.
- **Text splitters**: Break large documents into smaller chunks that will be retrievable individually and fit within a model's context window.
- **Embedding models**: An embedding model turns text into a vector of numbers so that texts with similar meaning land close together in that vector space.
- **Vector stores**: Specialized databases for storing and searching embeddings.
- **Retrievers**: A retriever is an interface that returns documents given an unstructured query.

RAG Architectures

RAG can be implemented in multiple ways, depending on your system's needs.

```
| Architecture | Description | Control | Flexibility | Latency | Example Use Case |
|---|---|---|---|---|---|
| 2-Step RAG | Retrieval always happens before generation. Simple and predictable | High | Low | Fast | FAQs, d
| Agentic RAG | An LLM-powered agent decides when and how to retrieve during reasoning | Low | High | Variable
| Hybrid | Combines characteristics of both approaches with validation steps | Medium | Medium | Variable | Dom
```

2-Step RAG

In 2-Step RAG, the retrieval step is always executed before the generation step. This architecture is straightforward and predictable, making it suitable for many applications where retrieval of relevant documents is a clear prerequisite for generating an answer.

Flow: User Question -> Retrieve Relevant Documents -> Generate Answer -> Return Answer to User.

This is the simplest and most explainable RAG architecture, and is well suited to documentation Q&A assistants where retrieval should always precede generation.

Agentic RAG

Agentic Retrieval-Augmented Generation combines the strengths of RAG with agent-based reasoning. Instead of retrieving documents before answering, an agent (powered by an LLM) reasons step-by-step and decides when and how to retrieve information during the interaction. The only thing an agent needs to enable RAG behavior is access to one or more tools that can fetch external knowledge, such as documentation loaders, web APIs, or database queries.

Hybrid RAG

Hybrid RAG combines characteristics of both 2-Step and Agentic RAG. It introduces intermediate steps such as query preprocessing, retrieval validation, and post-generation checks. Typical components include:

- **Query enhancement**: Modify the input question to improve retrieval quality — rewriting unclear queries, generating multiple variations, or expanding queries with additional context.
- **Retrieval validation**: Evaluate whether retrieved documents are relevant and sufficient. If not, the system may refine the query and retrieve again.
- **Answer validation**: Check the generated answer for accuracy, completeness, and alignment with source content.